



Variational Boosting for Physics-Informed Neural Networks

SCML 2026

Kaylee Vo Pavlos Protopapas

School of Engineering and Applied Sciences
Harvard University

September 14-17, 2026



1. Motivation
2. Method
3. Experiments
4. Conclusion
5. Appendix

Problem

Standard physics-informed neural networks (PINNs) solve

$$\min_{\theta} \mathcal{L}(u_{\theta}), \quad (1)$$

performing descent in the parameter space of a neural network. All scales and nonlinear interactions of the solution are entangled within a single model, often leading to ill-conditioning [1], spectral bias [2], and unstable training dynamics.

Solution

Boosting addresses this by **separating stages** for different scales. **Approach**: each small network is individually well-conditioned, and **sequential refinement** avoids the need to learn all scales simultaneously.

1. **Additive decomposition:** the solution is built from a sequence of correction networks. Each stage trains a weak learner whose converged correction satisfies a local orthogonality condition, equivalent to functional gradient descent.
2. **Second-order optimization:** because each weak learner is small, each stage admits full Newton or conjugate gradient updates.
3. **Transfer learning:** reusing information across correction stages improves convergence (fewer epochs or faster wall clock).

Stage $k = 0$ is a standard PINN trained on the full problem:

$$h_0 = \arg \min_{h \in \mathcal{U}_0} \mathcal{L}(h), \quad (2)$$

giving the initial approximation $u^{(0)} = h_0$. For $k \geq 1$, given $u^{(k-1)}$, we train a new weak learner, h_k , on the PDE residual. The update rule is

$$u^{(k)} = u^{(k-1)} + \alpha_k h_k, \quad (3)$$

where $\alpha_k > 0$ is a shrinkage hyperparameter. After K stages:

$$u^{(K)} = h_0 + \sum_{k=1}^K \alpha_k h_k. \quad (4)$$

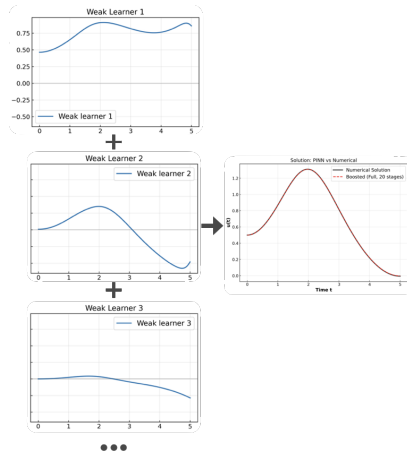


Figure 1: Weak learners combined additively to form solution u_θ .

- **Conjugate Gradient:** solves $(H + \gamma I)d = -g$ using only Hessian–vector products, avoiding explicit formation of the Hessian (H).
- **Newton's Method:** computes H explicitly, uses Tikhonov regularization when H is not positive-definite, and applies trust-region step-size control.
- Implementation: **Adam** → **second-order optimizer** ($\sim 90\%$ epochs are Adam, rest are CG/Newton). Lets Adam reach a well-conditioned region before exploiting local curvature.

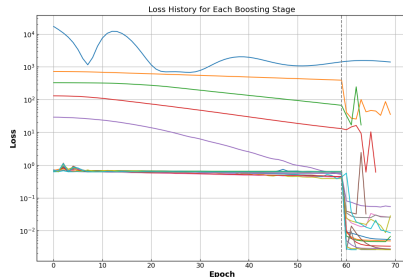
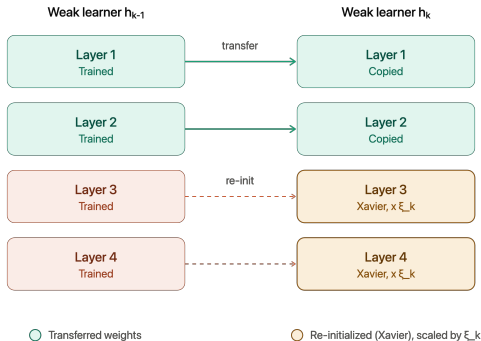


Figure 2: Loss curves per stage using Newton – nonlinear reaction-diffusion equation.



Metric	Monolithic PINN	Boosted PINN	
	L-BFGS	Adam + CG	Adam + Newton
Residual Norm	3.88×10^{-3}	5.43×10^{-3}	3.24×10^{-3}
MSE	3.77×10^{-4}	7.08×10^{-4}	1.44×10^{-4}
Relative L^2 Error	6.19×10^{-4}	9.64×10^{-4}	2.17×10^{-4}
Training Time (s)	0.31	7.03	187.72
Number of Iterations	132	738	407
Learning Rate	1.00	0.01	0.01
Collocation Points	1,000	1,000	1,000
Network Size	2,209	2,209	2,209

Table 1: Monolithic vs. Boosted PINN performance on the Duffing equation.



- Each h_k is initialized by transferring weights from h_{k-1} , with the final two layers re-initialized (Xavier) and rescaled by a stage-specific factor ξ_k .
- Removing transfer learning degrades performance, in some cases the PINN fails to converge.

Figure 3: Transfer learning setup for weak learners with hidden dimension equal to 3.

- Evaluation on a nonlinear coupled IVP: Lotka–Volterra.
- Trajectories initialized away from the equilibrium point form closed orbits, making this a challenging benchmark due to the oscillatory nature of the solutions.
- We find that boosted PINNs struggle with long time domains, as with standard PINNs.

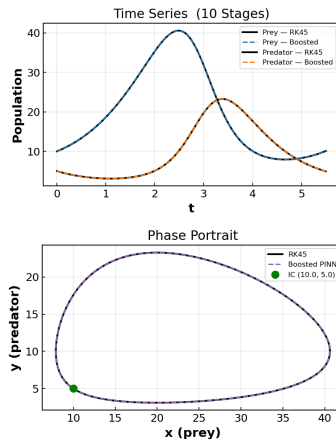


Figure 4:

Metric	Monolithic PINN		Boosted PINN
	Adam	L-BFGS	
Residual Norm	1.21×10^{-2}	2.50×10^{-1}	5.62×10^{-3}
MSE	1.74×10^{-2}	1.21×10^0	2.92×10^{-3}
Relative L^2 Error	9.52×10^{-4}	6.62×10^{-2}	1.60×10^{-4}
Training Time (s)	–	–	38.06
Iterations	–	–	26,400
Learning Rate	10^{-3}	1.0	10^{-3}
Collocation Points	1,000	1,000	1,000
Network Parameters	3,298	3,298	3,298

Table 2: Performance comparison on the Lotka–Volterra predator–prey system.

– indicates that the standard PINN did not converge; iteration count and training time are therefore not reported.

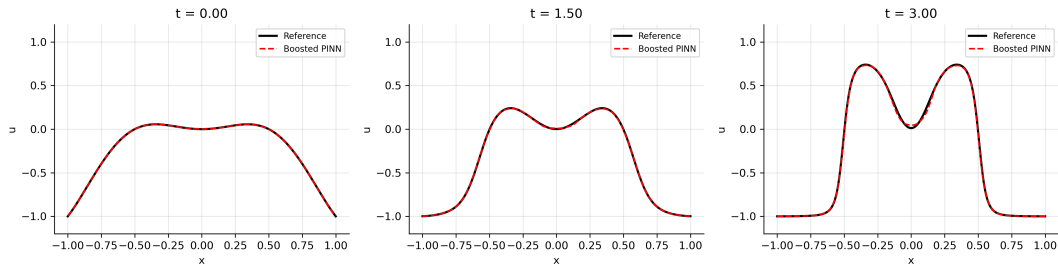


Figure 5: Boosted PINN solution 3D Comparison (Allen-Cahn with $D = 1.0 \times 10^{-4}$)

- Evaluation on Allen-Cahn PDE, with stiffness.
- Periodic boundary conditions combined with a small diffusion coefficient D cause the solution to develop sharp interfaces, inducing stiffness.

Optimizer	Monolithic PINN		Boosted PINN
	Adam	L-BFGS	Adam
Residual Norm	2.40×10^{-2}	3.66×10^{-3}	2.37×10^{-3}
MSE	1.89×10^{-1}	1.41×10^{-2}	7.10×10^{-3}
Relative L^2 Error	3.24×10^{-1}	2.60×10^{-2}	1.22×10^{-2}
Training Time (s)	–	–	499.44
Iterations	–	–	16,933
Learning Rate	$10^{-2} \rightarrow 1.91 \times 10^{-8}$	1.0	$10^{-2} \rightarrow 5.0 \times 10^{-3}$
Collocation Points	5,000	5,000	5,000
Network Parameters	8,641	8,641	8,641

Table 3: Model comparison for the Allen–Cahn equation with periodic boundary conditions ($D = 10^{-4}$).

– Did not converge to $\text{MSE} < 10^{-2}$; training time and iteration count not reported. Residual norm, MSE, and relative L^2 error are the best values achieved before termination.

Problem	Std. PINN MSE	Boosted MSE	Converges?
Duffing	3.77×10^{-4}	1.44×10^{-4}	Both
NRD ($\kappa=100$, stiff)	9.27×10^{-1}	8.01×10^{-3}	Boosted only
Van der Pol ($\mu=3$)	—	5.71×10^{-3}	Boosted only
Lotka–Volterra	1.74×10^{-2}	2.92×10^{-3}	Boosted only
Burgers' ($\nu = 0.002$)	9.27×10^{-1}	5.36×10^{-2}	Neither
Allen–Cahn ($D=0.0001$)	1.41×10^{-2}	7.10×10^{-3}	Boosted only

Table 4: Selected MSE comparison (best optimizer per model). “—” = did not converge.

- Compared to other ensemble methods [3], our method converges with **fewer iterations**. Our method also uses **smaller models** with performance on the same order of magnitude.
- Across ODE, coupled-ODE, and PDE benchmarks, the boosted PINN **matches or exceeds** standard PINN accuracy and converges where standard PINNs **fail outright**. Advantage pronounced in stiff regimes.

Problem	Method	Iterations	Wall-Clock
Duffing	Adam	$2.18\times$	$0.40\times$
	Adam + CG	$2.07\times$	$0.46\times$
	Adam + Newton	$3.76\times$	$0.02\times$
Allen–Cahn (nonstiff)	Adam	$1.00\times$	$1.26\times$

Table 5: Convergence metrics for boosted vs. standard PINN

Fixing epoch and parameter budget, boosted PINN converges but standard PINN fails on:

- NRD equation (stiff and nonstiff)
- Van der Pol equation (stiff and nonstiff)
- Lotka–Volterra coupled ODE system
- Allen–Cahn equation (stiff)

Problem	MSE (Transfer)	MSE (No Transfer)	Wilcoxon p
Duffing	2.04×10^{-3}	8.90×10^{-3}	0.3125
Lotka–Volterra	1.76×10^{-3}	7.91×10^{-1}	0.0042**
Burgers'	1.77×10^{-4}	2.91×10^{-4}	0.0420*

Table 6: Effect of transfer learning on MSE.

* $p < 0.05$.

** $p < 0.01$.

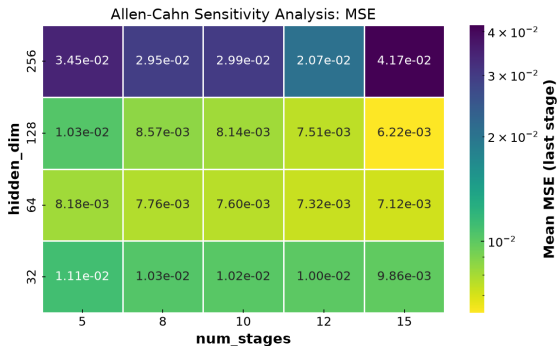


Figure 6: Sensitivity heatmap for Allen–Cahn

- **More stages / larger weak learners** → lower MSE for nonstiff problems (e.g. Lotka–Volterra), but for stiff PDEs (Allen–Cahn) an **intermediate** hidden size 64–128 performs best.
- If hidden dimension is too large, then you end up with standard PINNs issues. If too small, then estimation of functional gradient is inaccurate.

Takeaways

- We showed boosted PINNs converge on stiff equations where standard PINNs fail to, doing so with smaller models and smaller epoch budgets.
- Boosting generalizes: works on stiff/nonstiff, ODEs, ODE systems and PDEs.

Limitations

- There exist a level of stiffness that the boosted PINN cannot handle.
- Like standard PINNs, the boosted PINN also suffers from long temporal problems.

Future work

- Extending second-order optimization methods to stiff equations and PDEs.
- Addressing long-horizon problems.



Thank you! Questions?

kav418@g.harvard.edu



Scan for the preprint

- [1] Sifan Wang, Xinling Yu, and Paris Perdikaris. “When and why PINNs fail to train: A neural tangent kernel perspective”. In: *arXiv (Cornell University)* (July 2020). DOI: 10.48550/arxiv.2007.14527. URL: <http://arxiv.org/abs/2007.14527>.
- [2] Nasim Rahaman et al. “On the Spectral Bias of Neural Networks”. In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, Sept. 2019, pp. 5301–5310. URL: <https://proceedings.mlr.press/v97/rahaman19a.html>.
- [3] Rafael Bischof and Michael Kraus. “Mixture-of-Experts-Ensemble Meta-Learning for Physics-Informed Neural Networks”. In: Sept. 2022.

Boosted PINN Framework:

- The boosted PINN restricts the search at each stage k to a function class $\mathcal{U}_k \subset \mathcal{H}$, taken to be the set of functions representable by the k -th neural network.¹
- For notational ease, we have $\mathcal{H} = H^m(\Omega)$, where H^m is a Sobolev space. m represents the number of weak derivatives that must exist and be square-integrable.
- θ denotes the parameters of a neural network
- $\mathcal{L}$ denotes the physics loss (residual of the differential equation)

Second Order Optimization:

- For CG, $(H + \gamma I)d = -g$, γ is regularization hyperparameter. g is the gradient vector of the objective function (physics loss).
- γI is a type of (most common) Tikhonov regularization, which ensures H is positive-definite.

¹ k can be dropped if the network architecture is fixed across stages, which is the case for our paper. However, we leave the k index as is for generality.

Consider $\mathcal{L}(u) := \|F(u)\|_{L^2(\Omega)}^2$. The idealized gradient descent iteration in the Hilbert space $\mathcal{H}$ is

$$u_{k+1} = u_k - \alpha \nabla_{\mathcal{H}} \mathcal{L}(u_k) = u_k - 2\alpha F'(u_k)^* F(u_k), \quad (5)$$

where $F'(u_k)^*$ is the adjoint of the linearized residual operator (Fréchet derivative).

This is intractable: $u_k \in \mathcal{H}$ is infinite-dimensional; $F'(u_k)^*$ has no general closed form; and when $F'(u_k)^* F'(u_k)$ is ill-conditioned (stiff problems), α must be prohibitively small.

Boosted PINN: replace (5) with a finite-dimensional correction. At stage k , a network h_k is trained to approximate the descent direction $-2\alpha F'(u_k)^* F(u_k)$, giving

$$u_{k+1} = u_k + h_k, \quad (6)$$

turning an intractable infinite-dimensional iteration into a sequence of tractable finite-dimensional regressions.

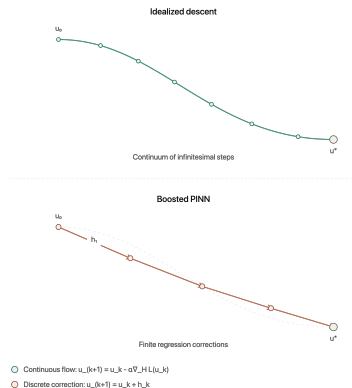


Figure 7: Idealized descent vs boosted PINN descent

The Lotka–Volterra system is characterized by the following:

$$\begin{cases} \frac{dx}{dt} = \alpha x - \beta xy, & x(0) = x_0 = 10.0, \\ \frac{dy}{dt} = -\gamma y + \delta xy, & y(0) = y_0 = 5.0, \end{cases} \quad (7)$$

where $x(t)$ denotes the prey population and $y(t)$ denotes the predator population.

- The parameter $\alpha > 0$ is the natural growth rate of the prey in the absence of predators, and $\beta > 0$ is the rate at which predators destroy prey, governed by the nonlinear interaction term xy .
- For the predator equation, $\gamma > 0$ is the natural death rate of predators in the absence of prey, and $\delta > 0$ is the rate at which predators increase by consuming prey.

The Allen–Cahn equation is given by

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + u - u^3, \quad x \in [-1, 1], \quad t \in [0, 3], \quad (8)$$

where D is the diffusion coefficient governing interface width. The stable equilibria of the reaction term $u - u^3$ are $u = \pm 1$, and for small D , the solution develops sharp interfaces between these two phases. The initial condition is

$$u(x, 0) = x^2 \cos(\pi x). \quad (9)$$

Periodic boundary conditions are imposed on the domain boundaries $x = -1$ and $x = 1$:

$$u(-1, t) = u(1, t), \quad \left. \frac{\partial u}{\partial x} \right|_{x=-1} = \left. \frac{\partial u}{\partial x} \right|_{x=1}, \quad t \in [0, 3]. \quad (10)$$

Let $L = x_{\max} - x_{\min} = 2$ be the period of the domain $[x_{\min}, x_{\max}] = [-1, 1]$. For a spatial coordinate x , define the phase angle

$$\varphi(x) = \frac{2\pi(x - x_{\min})}{L} = \pi(x + 1), \quad \varphi \in [0, 2\pi]. \quad (11)$$

The two-dimensional Fourier embedding is then

$$\phi(x) = \begin{pmatrix} \cos \varphi(x) \\ \sin \varphi(x) \end{pmatrix}, \quad (12)$$

and the augmented network input is

$$\mathbf{z}(x, t) = \begin{pmatrix} \cos(\pi(x + 1)) \\ \sin(\pi(x + 1)) \\ t \end{pmatrix} \in \mathbb{R}^3. \quad (13)$$

The stage 0 approximation is defined as

$$u^{(0)}(x, t) = \phi_0(\mathbf{z}(x, t); \theta_0), \quad (14)$$

where $\phi_0 : \mathbb{R}^3 \rightarrow \mathbb{R}$ is a fully connected neural network and θ_0 are its trainable parameters.

At $x = x_{\min} = -1$ and $x = x_{\max} = 1$, the phase angle (11) takes the values $\varphi(-1) = 0$ and $\varphi(1) = 2\pi$. Since $\cos$ and $\sin$ are 2π -periodic,

$$\mathbf{z}(-1, t) = \begin{pmatrix} 1 \\ 0 \\ t \end{pmatrix} = \mathbf{z}(1, t), \quad (15)$$

so that $u^{(0)}(-1, t) = u^{(0)}(1, t)$ for all t .

By the chain rule,

$$\frac{\partial u^{(0)}}{\partial x} = \frac{\partial \phi_0}{\partial \mathbf{z}} \cdot \frac{d\mathbf{z}}{dx} = \pi \frac{\partial \phi_0}{\partial \mathbf{z}} \cdot \begin{pmatrix} -\sin \varphi \\ \cos \varphi \\ 0 \end{pmatrix}. \quad (16)$$

Because $(-\sin 0, \cos 0) = (0, 1) = (-\sin 2\pi, \cos 2\pi)$, the input Jacobian $d\mathbf{z}/dx$ is identical at $x = -1$ and $x = 1$. With (15), this guarantees

$$\left. \frac{\partial u^{(0)}}{\partial x} \right|_{x=-1} = \left. \frac{\partial u^{(0)}}{\partial x} \right|_{x=1}, \quad (17)$$

satisfying the full periodic boundary condition (10) exactly. Each correction h_k at boosting stage k uses the identical embedding:

$$h_k(x, t) = \phi_k(\mathbf{z}(x, t); \theta_k), \quad (18)$$

so that the periodic conditions $h_k(-1, t) = h_k(1, t)$ and $\partial_x h_k|_{x=-1} = \partial_x h_k|_{x=1}$ hold for any choice of θ_k .